AI-ASSISTED CODING

ASSISGNMENT-13.4

P.Vishwateja

2303A52389

BATCH-44

TASK-1

PROMPT:

Refactor the following legacy Python code that uses an explicit loop to transform data. Use a more Pythonic approach such as list comprehension while preserving the same output.

CODE:

```
#TASK-1
values = [2, 4, 6, 8, 10]
doubled = []
for v in values:
    doubled.append(v * 2)
print(doubled)
```

OUTPUT:

```
3.exe "c:/Users/rahul/OneDrive/Desktop/AI ASSIST/lab-13.4.py"
[4, 8, 12, 16, 20]
```

OBSERVATION:

The original code used a loop and append operation to build a new list. The refactored code uses list comprehension, which performs the same operation in a single line. This improves code readability, conciseness, and execution efficiency while preserving the output.

TASK-2 :

PROMPT :

Refactor the following legacy Python code that constructs a sentence using repeated string concatenation.
Use a more efficient and readable Pythonic approach while keeping the same output.

CODE:

```python
#TASK-2
words = ["Refactoring", "with", "AI", "improves", "quality"]

message = " ".join(words)
print(message)
```

OUTPUT :

```
PS C:\Users\rahul\OneDrive\Desktop\AI ASSIST> & C:/Users/rahul/AppData/Local/Microsoft/WindowsApps/python3.1
3.exe "c:/Users/rahul/OneDrive/Desktop/AI ASSIST/lab-13.4.py"
Refactoring with AI improves quality
PS C:\Users\rahul\OneDrive\Desktop\AI ASSIST>
```

OBSERVATION:

The original code repeatedly concatenates strings inside a loop, which can be inefficient. The join() method combines all words in a list with a space separator in a single step. This approach is more readable, faster, and widely recommended in Python programming.

TASK-3

PROMPT :

Refactor the following legacy Python code that manually checks for a dictionary key before accessing it. Use a safer dictionary access method while keeping the same behaviour when the key is missing.

CODE:

```python
#TASK-3
config = {"host": "localhost", "port": 8080}
timeout = config.get("timeout")
if timeout:
    print(timeout)
else:
    print("Default timeout used")
```

OUTPUT:

```
PS C:\Users\rahul\OneDrive\Desktop\AI ASSIST> & C:/Users/rahul/AppData/Local/Microsoft/WindowsApps/python3.1
3.exe "c:/Users/rahul/OneDrive/Desktop/AI ASSIST/lab-13.4.py"
Default timeout used
PS C:\Users\rahul\OneDrive\Desktop\AI ASSIST>
```

OBSERVATION:

The original code explicitly checks whether the key exists in the dictionary. The refactored version uses get(), which safely retrieves the value without causing an error if

the key is missing. This approach simplifies the code and improves reliability and maintainability.

TASK-4

PROMPT :

Refactor the following Python code that uses multiple if–elif conditions to determine mathematical operations. Use a cleaner and more scalable mapping technique while preserving the same functionality and output.

CODE:

```
#TASK-4
action = "divide"
x, y = 10, 2
operations = {
    "add": lambda a, b: a + b,
    "subtract": lambda a, b: a - b,
    "multiply": lambda a, b: a * b,
    "divide": lambda a, b: a / b
}
result = operations.get(action, lambda a, b: None)(x, y)
print(result)
```

OUTPUT:

```
PS C:\Users\rahul\OneDrive\Desktop\AI ASSIST> & C:/Users/rahul/AppData/Local/Microsoft/WindowsApps/python3.1
3.exe "c:/Users/rahul/OneDrive/Desktop/AI ASSIST/lab-13.4.py"
5.0
PS C:\Users\rahul\OneDrive\Desktop\AI ASSIST>
```

OBSERVATION:

The original code used multiple conditional statements, which can become difficult to maintain as more operations are added. The refactored code uses a dictionary mapping approach, allowing operations to be easily extended and managed. This improves scalability, readability, and maintainability.

TASK-5

PROMPT :

Refactor the following legacy Python code that uses a manual loop to search for an item in a collection.Use a more concise and readable Pythonic approach while maintaining the same output behaviour.

CODE:

```
#TASK-5
inventory = ["pen", "notebook", "eraser", "marker"]
found = "eraser" in inventory
print("Item Available" if found else "Item Not Available")
```

OUTPUT:

```
PS C:\Users\rahul\OneDrive\Desktop\AI ASSIST> & C:/Users/rahul/AppData/Local/Microsoft/WindowsApps/python3.1
3.exe "c:/Users/rahul/OneDrive/Desktop/AI ASSIST/lab-13.4.py"
Item Available
PS C:\Users\rahul\OneDrive\Desktop\AI ASSIST>
```

OBSERVATION:

The original code uses a loop and a flag variable to search for an element. The refactored code uses the in operator, which directly checks if an element exists in a list. This makes the code shorter, clearer, and more Pythonic while maintaining the same functionality.